SR UNIVERSITY

**AI Assistant Coding**
**Assignment-14.4**

**Name of Student : V. Abhinav Batch : 41**
**Enrollment No. : 2303A52486**

## Task 1: Responsive Homepage Layout for a Startup Website

### Scenario

You are designing a basic homepage for a startup company that needs to look good on desktops, tablets, and mobile devices.

### Prompt:

*Generate a simple responsive homepage using HTML and CSS.*

*Requirements:*

*- Header with navigation links (Home, About, Services, Contact)*

*- Main content section with a heading and paragraph*

*- Footer with basic information*

*Use CSS Flexbox and media queries to make the layout responsive for:*

*- Mobile*

*- Tablet*

*- Desktop*

*Keep the code clean, simple, and beginner friendly.*

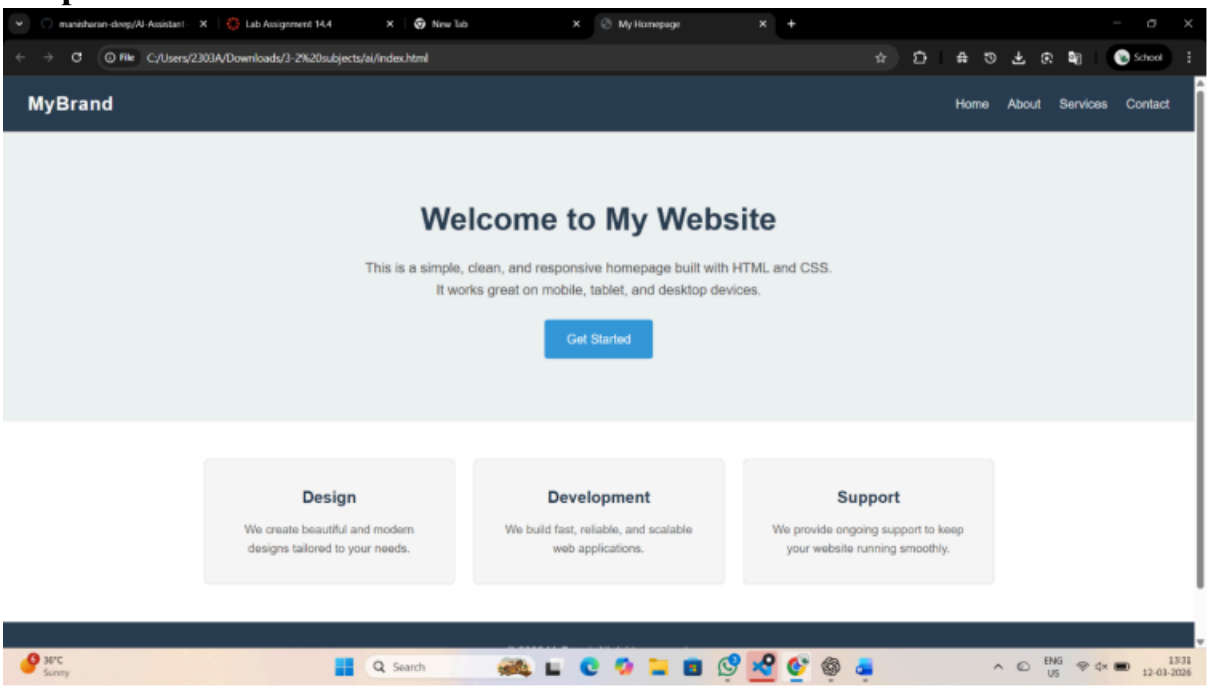
*Provide two files: index.html and styles.css.*

**Code:**

```html
index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My Homepage</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>

  <!-- Header -->
  <header class="header">
    <div class="logo">MyBrand</div>
    <nav class="nav">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </nav>
  </header>

  <!-- Main Content -->
  <main class="main">
    <section class="hero">
      <h1>Welcome to My Website</h1>
      <p>
        This is a simple, clean, and responsive homepage built with HTML and CSS.
        It works great on mobile, tablet, and desktop devices.
      </p>
      <a href="#" class="btn">Get Started</a>
    </section>
```

```html
<!-- Main Content -->
<main class="main">
  <section class="hero">
    <h1>Welcome to My Website</h1>
    <p>
      This is a simple, clean, and responsive homepage built with HTML and CSS.
      It works great on mobile, tablet, and desktop devices.
    </p>
    <a href="#" class="btn">Get Started</a>
  </section>

  <section class="cards">
    <div class="card">
      <h2>Design</h2>
      <p>We create beautiful and modern designs tailored to your needs.</p>
    </div>
    <div class="card">
      <h2>Development</h2>
      <p>We build fast, reliable, and scalable web applications.</p>
    </div>
    <div class="card">
      <h2>Support</h2>
      <p>We provide ongoing support to keep your website running smoothly.</p>
    </div>
  </section>
</main>

<!-- Footer -->
<footer class="footer">
  <p>&copy; 2026 MyBrand. All rights reserved.</p>
  <p>contact@mybrand.com | +1 (555) 123-4567</p>
</footer>

/body>
```

**Output:**




**Explanation:**

A responsive homepage layout ensures that the website works properly on different devices such as mobile phones, tablets, and desktops. Using HTML provides a clear structure for the header, main content, and footer sections, making the page organized and easy to understand. CSS with media queries helps adjust the layout according to screen size, improving user experience and accessibility. This approach makes the website flexible, visually consistent, and suitable for modern web development standards.

**Task 2: Interactive Call-to-Action Button**

**Scenario:**

A marketing page requires a call-to-action button that responds when

users interact with it.

**Prompt:**

*Create a simple web page with an interactive Call-to-Action button using HTML, CSS, and JavaScript.*

*Requirements:*

*- Add a button labeled "Click Me".*

*- When the button is clicked, display a message or alert using JavaScript.*

*- Do not reload the page when the button is clicked.*

*- Add clear and meaningful comments in the JavaScript code explaining the logic.*

*Provide the complete code in one HTML file.*

*Keep the code simple and beginner friendly.*

**Code:**

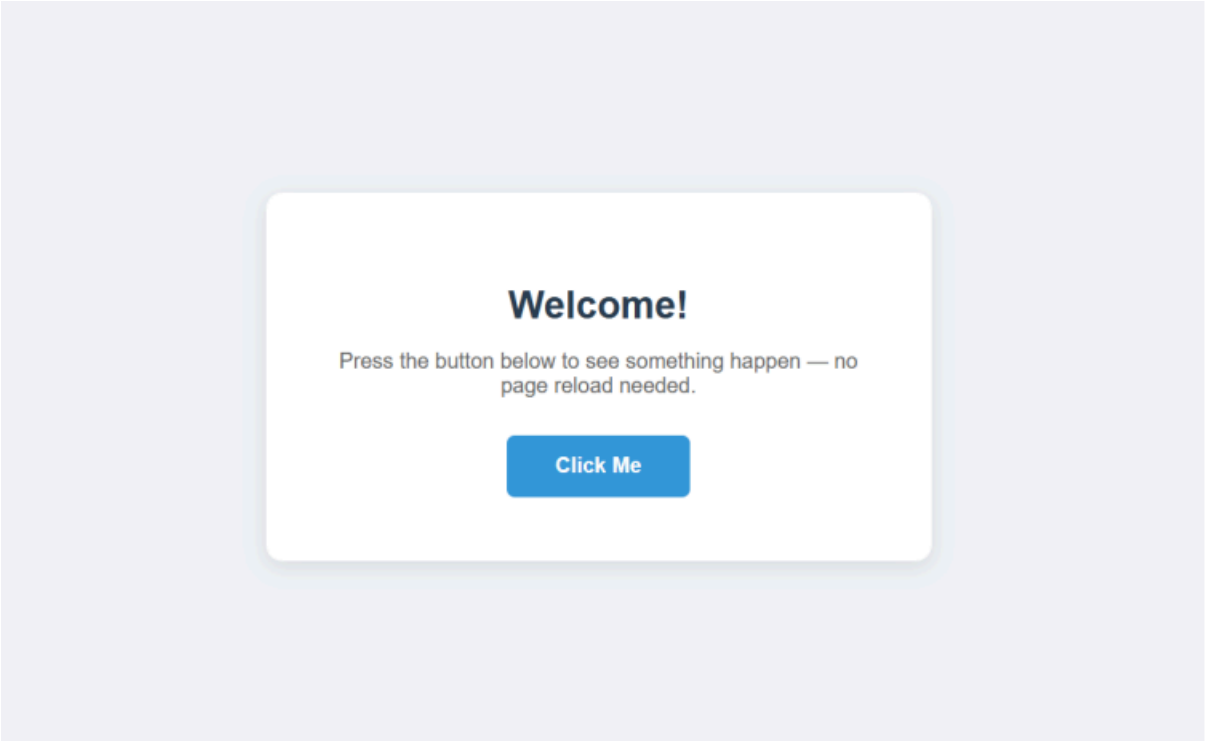
cta-button.html > ...

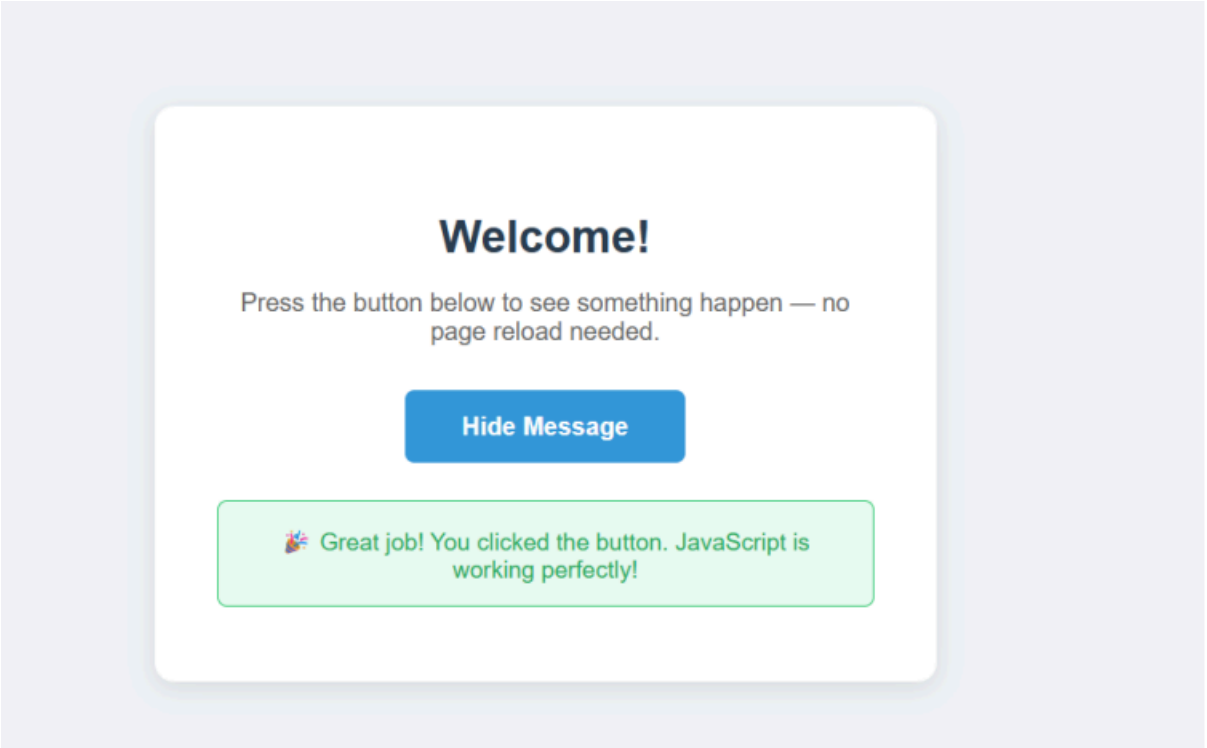
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Interactive CTA Button</title>
  <style>
    /* — Base styles — */
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f4f8;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    /* — Card container — */
    .card {
      background-color: #fff;
      border-radius: 12px;
      box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);
      padding: 48px 40px;
      text-align: center;
      max-width: 420px;
      width: 90%;
    }

    .card h1 {
      font-size: 1.8rem;
      color: #2c3e50;
      margin-bottom: 12px;
    }

    .card p {
      color: #666;
```

```
    .btn {
      padding: 14px 36px;
      font-size: 1rem;
      font-weight: bold;
      color: #fff;
      background-color: #3498db;
      border: none;
      border-radius: 6px;
      cursor: pointer;
      transition: background-color 0.2s, transform 0.1s;
    }

    .btn:hover {
      background-color: #2980b9;
    }

    .btn:active {
      transform: scale(0.97);
    }

    /* — Message box (hidden by default) — */
    .message {
      display: none;                /* hidden until button is clicked */
      margin-top: 24px;
      padding: 14px 20px;
      background-color: #eafaf1;
      border: 1px solid #2ecc71;
      border-radius: 6px;
      color: #27ae60;
      font-size: 0.95rem;
    }
  </style>
</head>
<body>
```

```
    // Check if the message is currently hidden (display is "none" or empty)
    if (message.style.display === "none" || message.style.display === "") {

      // Show the message by changing its CSS display to "block"
      message.style.display = "block";

      // Change the button label to indicate it can be clicked again to hide
      button.textContent = "Hide Message";

    } else {

      // If the message is already visible, hide it again
      message.style.display = "none";

      // Reset the button label back to the original text
      button.textContent = "Click Me";
    }
  });
</script>

/body>
/html>
```

**Output:**


Welcome!
Press the button below to see something happen — no page reload needed.
Click Me


Welcome!
Press the button below to see something happen — no page reload needed.
Hide Message
🎉 Great job! You clicked the button. JavaScript is working perfectly!

**Explanation:**

An interactive Call-to-Action (CTA) button improves user engagement by allowing users to interact directly with the webpage. HTML is used to create the button element, while JavaScript handles the click event and displays a message or alert without reloading the page. Adding comments in the JavaScript code makes the logic easier to understand and maintain. This approach demonstrates basic event handling and enhances the overall user experience on the website**.**

**Task 3: Contact Form with Client-Side Validation**

**Scenario**

- **You are developing a contact form for a company website to collect**
- **user queries safely and correctly**

**Prompt:**

*Create a simple Contact Form using HTML, CSS, and JavaScript.*

*Requirements:*

*- Form fields: Name, Email, and Message.*

*- Use JavaScript for client-side validation.*

*- Prevent form submission if fields are empty.*

*- Validate that the email format is correct.*

*- Show clear inline error messages for invalid inputs.*

*- Show a success message when the form is valid.*

*- Do not reload the page on validation.*

*Keep the code simple, clean, and beginner friendly. Provide the complete code in one HTML file.*

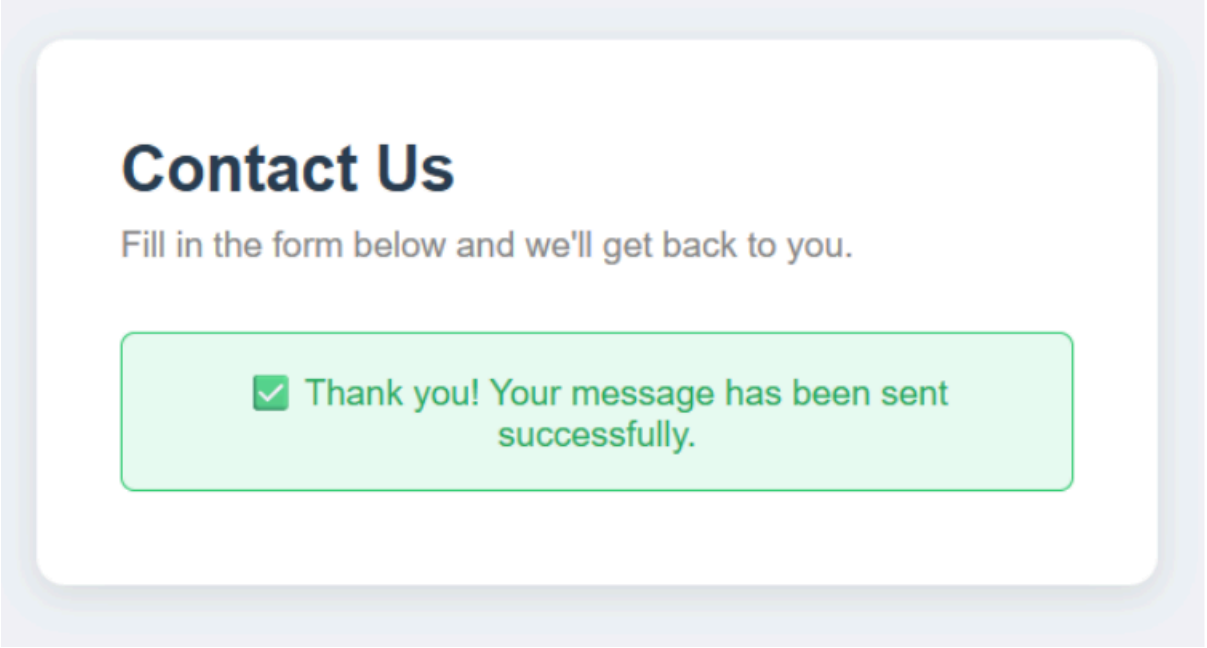
**Code:**

```
contact-form.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Contact Form</title>
  <style>
    /* — Base styles — */
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: Arial, sans-serif;
      background-color: #f0f4f8;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      padding: 24px;
    }

    /* — Form card — */
    .form-card {
      background-color: #fff;
      border-radius: 12px;
      box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);
      padding: 40px 36px;
      width: 100%;
      max-width: 480px;
    }

    .form-card h1 {
      font-size: 1.7rem;
      color: #2c3e50;
```

```
    <form id="contactForm" novalidate>
      <!-- Email field -->
      <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" placeholder="john@example.com" />
        <span class="error-msg" id="emailError">Please enter a valid email address.</span>
      </div>

      <!-- Message field -->
      <div class="form-group">
        <label for="message">Message</label>
        <textarea id="message" placeholder="Write your message here..."></textarea>
        <span class="error-msg" id="messageError">Please enter a message.</span>
      </div>

      <button type="submit" class="btn-submit">Send Message</button>
    </form>

    <!-- Success message shown after valid submission -->
    <div class="success-msg" id="successMsg">
      ✅ Thank you! Your message has been sent successfully.
    </div>
  </div>

  <script>
    // — Select all form elements —
    const form         = document.getElementById("contactForm");
    const nameInput    = document.getElementById("name");
    const emailInput   = document.getElementById("email");
    const msgInput     = document.getElementById("message");
    const nameError    = document.getElementById("nameError");
    const emailError   = document.getElementById("emailError");
    const msgError     = document.getElementById("messageError");
    const successMsg   = document.getElementById("successMsg");
```

```
      } else {
        clearError(nameInput, nameError);
      }

      // — Validate Email: must not be empty AND must match email format —
      if (emailValue === "" || !isValidEmail(emailValue)) {
        showError(emailInput, emailError);
        isValid = false;
      } else {
        clearError(emailInput, emailError);
      }

      // — Validate Message: must not be empty —
      if (msgValue === "") {
        showError(msgInput, msgError);
        isValid = false;
      } else {
        clearError(msgInput, msgError);
      }

      // — If all fields are valid, show success message —
      if (isValid) {
        // Hide the form so the success message takes its place
        form.style.display = "none";

        // Show the success confirmation message
        successMsg.style.display = "block";
      }
    });
  </script>

</body>
</html>
```

**Output:**



**Explanation:**

The legacy code manually checked key presence if "timeout" in config

Issues:

Verbose structure. Not scalable for multiple keys. Increases cognitive load.

Risk of KeyError if incorrectly accessed. The improved version uses:

config.get("timeout", "Default timeout used")

Advantages:

Built-in safe accessor. Handles missing keys gracefully. Reduces conditional branching. More concise and expressive. This improves robustness and follows defensive programming principles.

Behavior remains identical:

If key exists → return value. If key missing → return default message.

**Task 4: Dynamic List Management for a Product Page**

**Scenario**

**A product listing page needs to add or remove items dynamically**

**without refreshing the page.**

**Prompt:**

*Create a simple web page using HTML, CSS, and JavaScript for dynamic list management.*

*Requirements:*

*- Display a list of items using an HTML unordered list.*

*- Add an input field and an "Add" button to insert a new item into the list.*

*- Add a "Remove" button to remove the last item from the list.*

*- Use JavaScript DOM manipulation to update the list dynamically.*

*- Ensure items are added and removed instantly without reloading the page.*

*- Add clear comments in the JavaScript code explaining the logic.*

*Provide the complete code in one HTML file. Keep it simple and beginner friendly.*

**Code:**

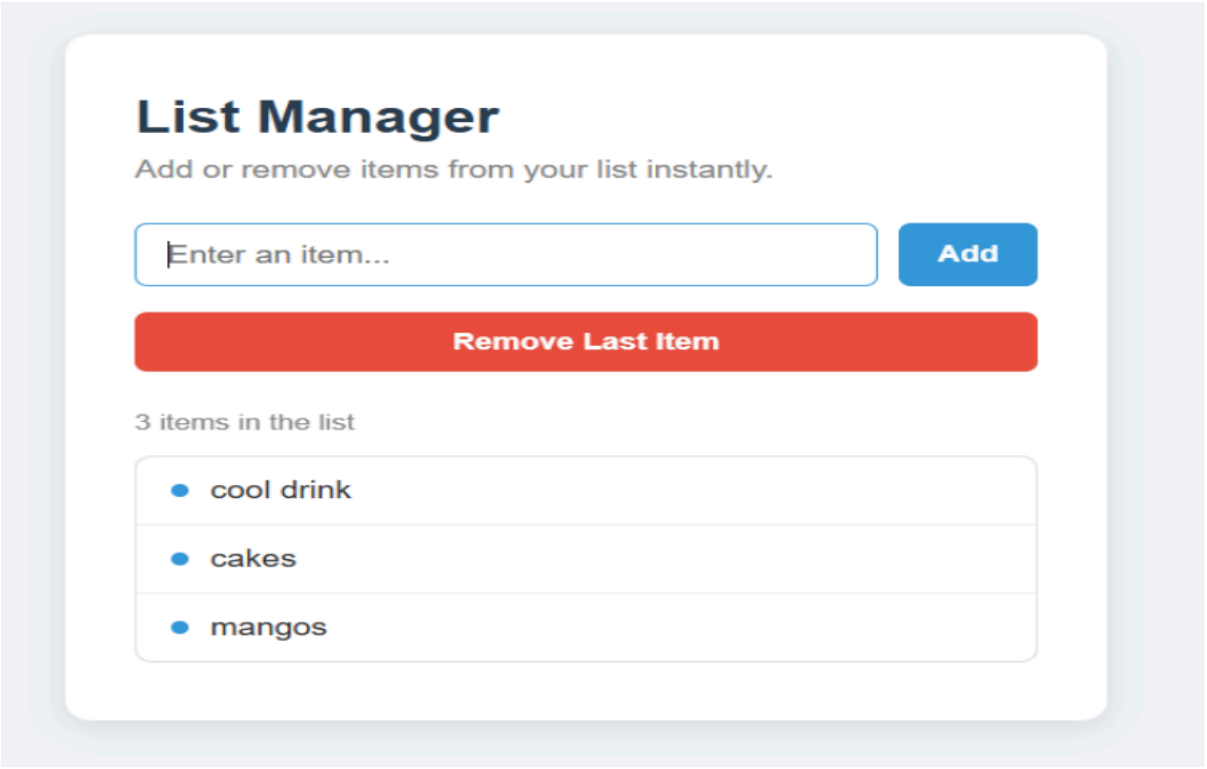
list-manager.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dynamic List Manager</title>
  <style>
    /* — Base styles — */
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: Arial, sans-serif;
      background-color: #f0f4f8;
      display: flex;
      justify-content: center;
      align-items: flex-start;
      min-height: 100vh;
      padding: 48px 24px;
    }

    /* — Card container — */
    .card {
      background-color: #fff;
      border-radius: 12px;
      box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);
      padding: 36px 32px;
      width: 100%;
      max-width: 480px;
    }

    .card h1 {
      font-size: 1.7rem;
      color: #2c3e50;
```

```
.card p.subtitle {
  color: #888;
  font-size: 0.95rem;
  margin-bottom: 24px;
}

/* — Input row — */
.input-row {
  display: flex;
  gap: 10px;
  margin-bottom: 16px;
}

.input-row input {
  flex: 1;
  padding: 10px 14px;
  font-size: 0.95rem;
  border: 1px solid #ccc;
  border-radius: 6px;
  outline: none;
  transition: border-color 0.2s;
}

.input-row input:focus {
  border-color: #3498db;
}

/* — Buttons — */
.btn {
  padding: 10px 18px;
  font-size: 0.9rem;
  font-weight: bold;
  border: none;
```

```
  // Update the item count and button state
  updateCount();
}

// — Event listeners —

// Add button click → add item
addBtn.addEventListener("click", addItem);

// Remove button click → remove last item
removeBtn.addEventListener("click", removeLastItem);

// Allow pressing Enter in the input field to add an item quickly
input.addEventListener("keydown", function (event) {
  if (event.key === "Enter") {
    addItem();
  }
});

// Hide the error message as soon as the user starts typing
input.addEventListener("input", function () {
  if (input.value.trim() !== "") {
    errorMsg.style.display = "none";
  }
});

// — Initialise count on page load —
updateCount();
</script>

</body>
</html>
```

**Output:**




**Explanation:**

Dynamic list management improves user interaction by allowing items to be added or removed from a webpage instantly without refreshing the page. HTML is used to create the list structure, while JavaScript manipulates the DOM to update the list dynamically when users click the Add or Remove buttons. This demonstrates basic DOM operations, event handling, and improves the responsiveness of the webpage.

**Task 5: Modal Popup for User Notifications**

**Scenario:**

You are building a user notification popup for announcements or alerts

on a website.

**Prompt:**

*Create a simple Modal Popup notification using HTML, CSS, and JavaScript.*

*Requirements:*

*- Add a button labeled "Open Modal".*

*- When the button is clicked, display a modal popup in the center of the page.*

*- The modal should have a semi-transparent background overlay.*

*- Include a close button inside the modal.*

*- Allow the modal to close when the close button or the overlay is clicked.*

*- Use JavaScript to handle opening and closing of the modal.*

*- Keep the code simple, clean, and well commented.*

*Provide the complete code in one HTML file.*

**Code:**

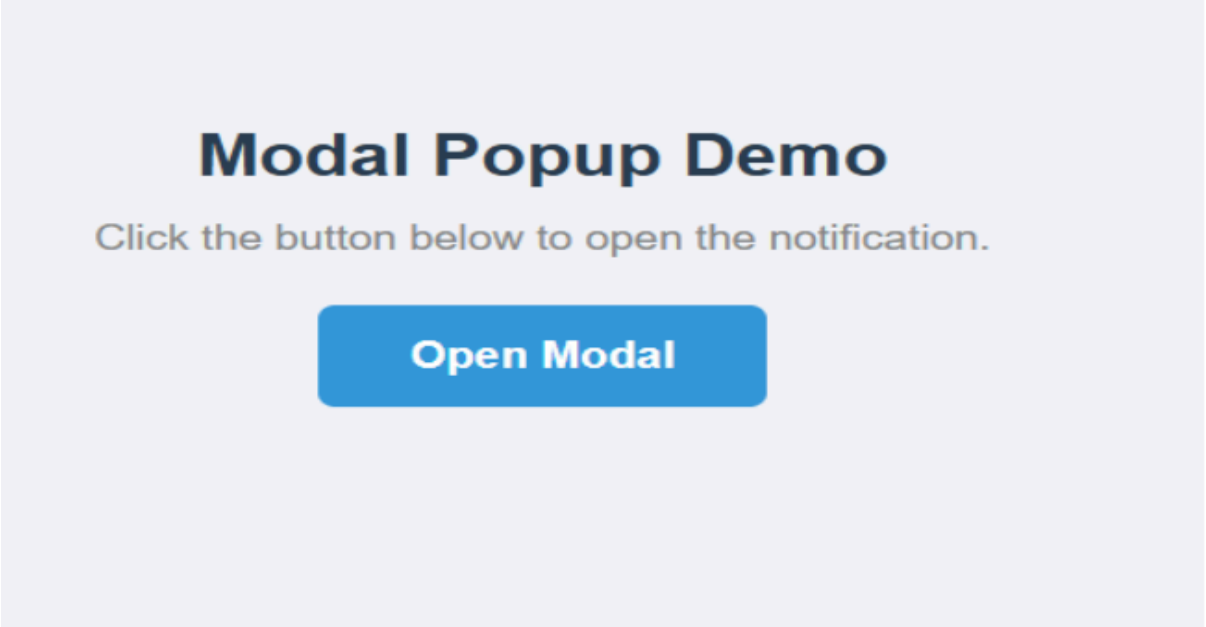
modal-popup.html > ...

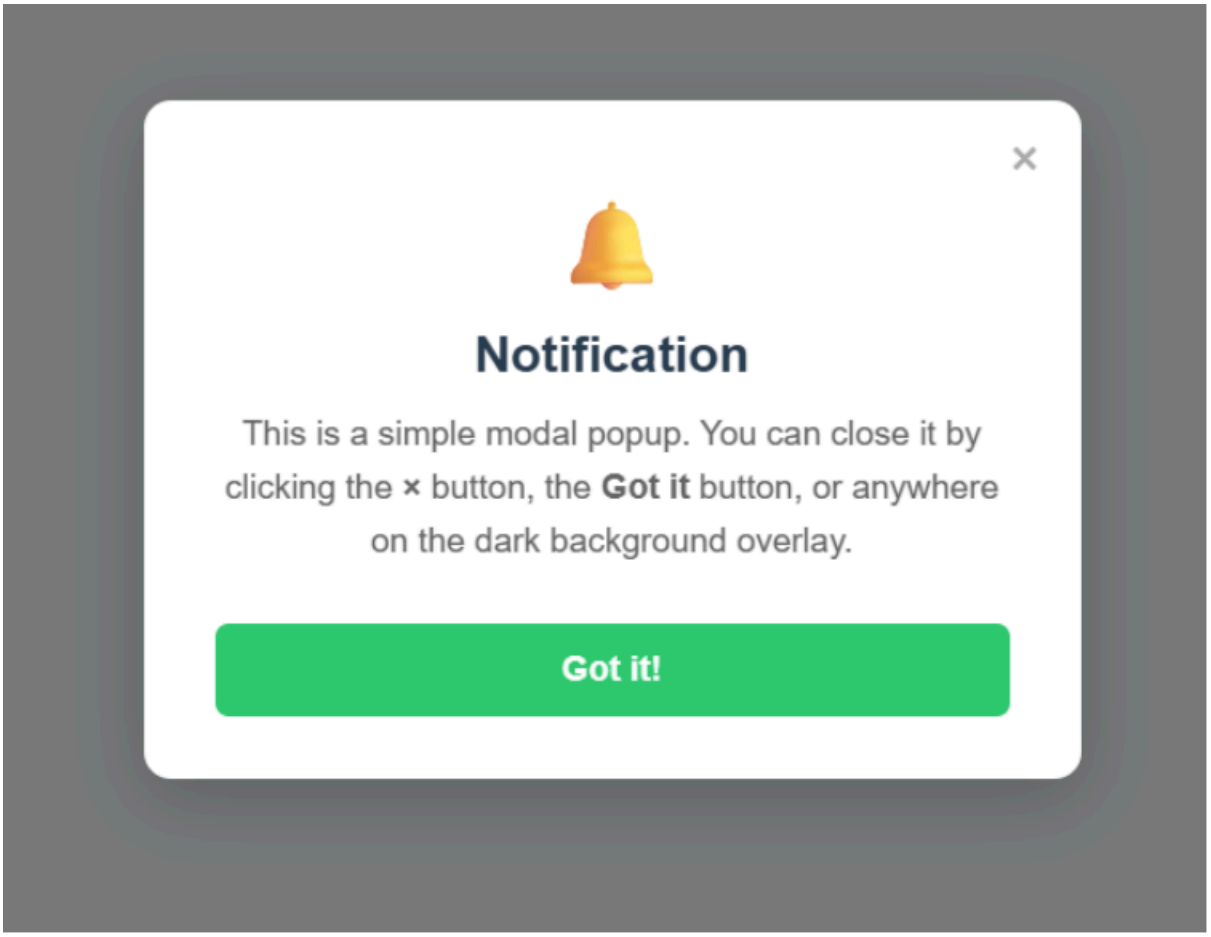
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Modal Popup</title>
  <style>
    /* — Base styles — */
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: Arial, sans-serif;
      background-color: #f0f4f8;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      gap: 12px;
    }

    body h1 {
      color: #2c3e50;
      font-size: 1.6rem;
    }

    body p {
      color: #888;
      font-size: 0.95rem;
      margin-bottom: 8px;
    }

    /* — Open Modal button — */
```

```html
<!-- Button that triggers the modal -->
<button class="btn-open" id="openBtn">Open Modal</button>

<!-- Overlay: clicking it closes the modal -->
<div class="overlay" id="overlay">

  <!-- Modal box: clicks inside here do NOT close the modal -->
  <div class="modal" id="modal">

    <!-- Close (×) button in the corner -->
    <button class="btn-close" id="closeBtn" aria-label="Close modal">&times;</button>

    <!-- Modal content -->
    <div class="modal-icon">🔔</div>
    <h2>Notification</h2>
    <p>
      This is a simple modal popup. You can close it by clicking the
      <strong>&times;</strong> button, the <strong>Got it</strong> button,
      or anywhere on the dark background overlay.
    </p>

    <!-- Dismiss button inside the modal -->
    <button class="btn-dismiss" id="dismissBtn">Got it!</button>

  </div>
</div>

<script>
  // ── Select elements from the DOM ──
  const openBtn    = document.getElementById("openBtn");
  const overlay    = document.getElementById("overlay");
  const closeBtn   = document.getElementById("closeBtn");
  const dismissBtn = document.getElementById("dismissBtn");
  const modal      = document.getElementById("modal");
```

```
  // — Open the modal when the "Open Modal" button is clicked —
  openBtn.addEventListener("click", openModal);

  // — Close the modal when the x button is clicked —
  closeBtn.addEventListener("click", closeModal);

  // — Close the modal when the "Got it!" dismiss button is clicked —
  dismissBtn.addEventListener("click", closeModal);

  // — Close the modal when the overlay (dark background) is clicked —
  // We check that the click target is the overlay itself, NOT the modal box,
  // so that clicking inside the modal does not accidentally close it.
  overlay.addEventListener("click", function (event) {
    if (event.target === overlay) {
      closeModal();
    }
  });

  // — Allow pressing Escape key to close the modal —
  document.addEventListener("keydown", function (event) {
    if (event.key === "Escape") {
      closeModal();
    }
  });
</script>

</body>
</html>
```

**Output:**






**Explanation:**

A modal popup is useful for displaying important notifications, announcements, or alerts without navigating away from the current page. HTML creates the modal structure, CSS provides styling and a semi-transparent overlay to focus user attention, and JavaScript controls opening and closing actions. Allowing multiple ways to close the modal improves usability and provides a better user experience.